單位換算器—AI 對話

A1143344 張鈞甯

1. 詢問 JComboBox 的程式如何寫

要使用java的JComboBox的話要怎麼寫呢

在 Java Swing 中，`JComboBox` 是一個下拉式選單元件，允許使用者從一組選項中選擇一個項目。

以下是使用 `JComboBox` 的基本步驟、完整程式碼範例以及常見的操作方法。

1. 完整程式碼範例

這是一個簡單的視窗程式，包含一個下拉選單和一個標籤（JLabel），當你選擇不同的選項時，標籤會即時顯示你所選的內容：

Java

```java
import javax.swing.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class JComboBoxExample {
    public static void main(String[] args) {
        // 1. 建立視窗
        JFrame frame = new JFrame("JComboBox 範例");
        frame.setSize(400, 200);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLayout(null); // 使用絕對布局方便範例定位

        // 2. 準備下拉選單的選項資料
        String[] fruits = { "蘋果", "香蕉", "橘子", "葡萄", "西瓜" };

        // 3. 建立 JComboBox 物件
        JComboBox<String> comboBox = new JComboBox<>(fruits);
        comboBox.setBounds(50, 50, 150, 30);

        // 4. 建立標籤用來顯示選取的結果
        JLabel label = new JLabel("你選擇的是: 蘋果");
        label.setBounds(50, 100, 250, 30);

        // 5. 加入事件監聽器（處理選擇變更）
        comboBox.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                // 取得目前被選取的項目
                String selected = (String) comboBox.getSelectedItem();
                label.setText("你選擇的是: " + selected);
            }
        });
```

```
        // 6. 將元件加入視窗並顯示
        frame.add(comboBox);
        frame.add(label);
        frame.setVisible(true);
    }
}
```

2. 常見操作與常用方法

除了初始化時直接放入陣列，你還可以在程式執行過程中對 `JComboBox` 進行動態操作：

- **新增項目：**

```java
comboBox.addItem("草莓");
```

- **移除項目：**

```java
comboBox.removeItem("香蕉"); // 依物件移除
comboBox.removeItemAt(0);    // 依索引移除
```

- **取得目前選取的項目：**

```java
String selectedText = (String) comboBox.getSelectedItem();
int selectedIndex = comboBox.getSelectedIndex(); // 取得索引值（從 0 開始）
```

- **設定預設選取的項目：**

```java
comboBox.setSelectedIndex(2); // 選擇第 3 個項目
comboBox.setSelectedItem("葡萄"); // 依內容選擇
```

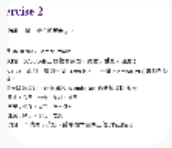
3. 進階：讓 JComboBox 變成可編輯的

預設情況下，使用者只能從現有的選項中選擇。如果你想讓使用者能夠**自行輸入**新的選項，可以設定為可編輯：

Java

```
comboBox.setEditable(true);
```

2.詢問 AI 單位換算器的程式應如何寫



幫我寫這個java程式
我目前寫到這裡：
import javax.swing.*;
import java.awt.*;

根據你提供的題目要求與目前的程式碼架構，我幫你完成了這個「單位換算器」的完整程式。

完整程式碼

Java

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class Ch17_hw3 {
    static JFrame frm = new JFrame("單位換算器");

    public static void main(String args[]) {
        BorderLayout border = new BorderLayout();
        frm.setLayout(border);
        frm.setSize(480, 280);
        frm.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```java
// 1. NORTH：JComboBox 選擇換算類型
String[] types = {"長度", "重量", "溫度"};
JComboBox<String> typeBox = new JComboBox<>(types);

JPanel northPanel = new JPanel();
northPanel.add(new JLabel("選擇換算類型："));
northPanel.add(typeBox);
frm.add(northPanel, BorderLayout.NORTH);

// 2. CENTER：兩列，每列一個 JTextField + 一個 JComboBox
JPanel centerPanel = new JPanel(new GridLayout(2, 1, 5, 5));

// 第一列：來源數值與來源單位
JPanel row1 = new JPanel();
JTextField tfSource = new JTextField(10);
JComboBox<String> cbSource = new JComboBox<>();
row1.add(new JLabel("來源數值："));
row1.add(tfSource);
row1.add(cbSource);

// 第二列：目標數值（顯示結果）與目標單位
JPanel row2 = new JPanel();
JTextField tfTarget = new JTextField(10);
tfTarget.setEditable(false); // 結果欄位設為唯讀
JComboBox<String> cbTarget = new JComboBox<>();
row2.add(new JLabel("換算結果："));
row2.add(tfTarget);
row2.add(cbTarget);

centerPanel.add(row1);
centerPanel.add(row2);
frm.add(centerPanel, BorderLayout.CENTER);

// 初始化單位下拉選單的選項
updateUnits(typeBox, cbSource, cbTarget);
```

```java
// 切換換算類型時，動態更新單位 JComboBox 的選項
typeBox.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        updateUnits(typeBox, cbSource, cbTarget);
    }
});

// 3. SOUTH：「換算」按鈕
JButton btnCalc = new JButton("換算");
JPanel southPanel = new JPanel();
southPanel.add(btnCalc);
frm.add(southPanel, BorderLayout.SOUTH);
```

```
        // 點擊換算按鈕的邏輯
        btnCalc.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                try {
                    double val = Double.parseDouble(tfSource.getText());
                    String type = (String) typeBox.getSelectedItem();
                    String srcUnit = (String) cbSource.getSelectedItem();
                    String tgtUnit = (String) cbTarget.getSelectedItem();

                    double result = convert(type, srcUnit, tgtUnit, val);
                    tfTarget.setText(String.format("%.4f", result));
                } catch (NumberFormatException ex) {
                    JOptionPane.showMessageDialog(frm, "請輸入有效的數字！", "錯誤", JOp
                }
            }
        });

        frm.setVisible(true);
    }
```

```
    // 根據選擇的類型更新單位選單
    private static void updateUnits(JComboBox<String> typeBox, JComboBox<String> cbS
        String selectedType = (String) typeBox.getSelectedItem();
        cbSource.removeAllItems();
        cbTarget.removeAllItems();

        String[] units;
        if ("長度".equals(selectedType)) {
            units = new String[]{"公尺", "公分", "英吋", "英尺"};
        } else if ("重量".equals(selectedType)) {
            units = new String[]{"公斤", "公克", "磅", "盎司"};
        } else { // 溫度
            units = new String[]{"攝氏", "華氏", "克氏"};
        }

        for (String u : units) {
            cbSource.addItem(u);
            cbTarget.addItem(u);
        }
        if (cbTarget.getItemCount() > 1) {
            cbTarget.setSelectedIndex(1); // 預設目標選第二個項目
        }
    }
```

```
    // 換算核心邏輯
    private static double convert(String type, String src, String tgt, double val) {
        if (type.equals("長度")) {
            double meters = val;
            switch (src) {
                case "公尺": meters = val; break;
                case "公分": meters = val * 0.01; break;
                case "英吋": meters = val * 0.0254; break;
                case "英尺": meters = val * 0.3048; break;
            }
```

```
            switch (tgt) {
                case "公尺": return meters;
                case "公分": return meters / 0.01;
                case "英吋": return meters / 0.0254;
                case "英尺": return meters / 0.3048;
            }
        } else if (type.equals("重量")) {
            double kg = val;
            switch (src) {
                case "公斤": kg = val; break;
                case "公克": kg = val * 0.001; break;
                case "磅": kg = val * 0.45359237; break;
                case "盎司": kg = val * 0.0283495231; break;
            }
            switch (tgt) {
                case "公斤": return kg;
                case "公克": return kg / 0.001;
                case "磅": return kg / 0.45359237;
                case "盎司": return kg / 0.0283495231;
            }
        } else if (type.equals("溫度")) {
            double celsius = val;
            switch (src) {
                case "攝氏": celsius = val; break;
                case "華氏": celsius = (val - 32) * 5.0 / 9.0; break;
                case "克氏": celsius = val - 273.15; break;
            }
            switch (tgt) {
                case "攝氏": return celsius;
                case "華氏": return celsius * 9.0 / 5.0 + 32;
                case "克氏": return celsius + 273.15;
            }
        }
        return val;
    }
}
```

設計重點說明：

1. **版面配置（`BorderLayout`）：**
   - **NORTH**：放置選擇類型的 `JComboBox`。
   - **CENTER**：使用 `GridLayout(2, 1)` 放置上下兩列，每列包含一個 `JLabel`、輸入/結果用的 `JTextField` 以及單位 `JComboBox`。 image_0ab81c.png
   - **SOUTH**：放置「換算」按鈕。 image_0ab81c.png
2. **動態更新單位選項**：透過 `updateUnits` 方法，當上方選單切換時（長度/重量/溫度），會自動清空並重新載入對應的單位選項到來源與目標的 `JComboBox` 中。 image_0ab81c.png
3. **換算邏輯**：
   - 長度與重量先統一換算成基準單位（公尺、公斤），再轉成目標單位。
   - 溫度則轉換為攝氏後再對應轉換。
   - 第二個 `JTextField` 設定為 `setEditable(false)`，專門用來顯示計算後的結果。 image_0ab81c.png

3.詢問 AI 程式中不懂的地方

updateUnits、convert還有
for (String u : units) {
cbSource.addItem(u);
cbTarget.addItem(u);

這幾個部分是讓這個「單位換算器」能夠**動態變更選項**以及**進行數值計算**的核心邏輯。以下為你逐一拆解說明：

1. `for (String u : units) { ... }` 迴圈

這是一個 Java 的**加強型 for 迴圈（Enhanced for-loop）**，用來走訪（遍歷）陣列中的每一個元素。

- **白話文解釋**：
  假設 `units` 陣列裡面裝的是 `["公尺", "公分", "英吋", "英尺"]`。這個迴圈會把陣列裡的元素一個一個拿出來，暫時存放在變數 `u` 裡面。

- **程式碼做了什麼：**

```java
for (String u : units) {
    cbSource.addItem(u); // 將單位加入「來源單位」下拉選單
    cbTarget.addItem(u); // 將單位加入「目標單位」下拉選單
}
```

它會依序把「公尺」、「公分」、「英吋」、「英尺」同時加進來源的 `JComboBox`（`cbSource`）和目標的 `JComboBox`（`cbTarget`）裡，讓使用者可以點選這些單位。

2. `updateUnits` 方法

這是我們自訂的一個**工具方法（Function）**，主要功能是：**根據上方使用者選擇的大分類（長度、重量、溫度），來動態改變下方兩個單位選單裡的選項。** image_0ab81c.png

- **程式碼與運作流程：**

```java
private static void updateUnits(JComboBox<String> typeBox, JComboBox<String> c
    // 1. 取得目前使用者在最上方大分類選了什麼（例如：「長度」）
    String selectedType = (String) typeBox.getSelectedItem();

    // 2. 先清空來源和目標下拉選單原本舊有的選項，避免選項越疊越多
    cbSource.removeAllItems();
    cbTarget.removeAllItems();

    // 3. 根據大分類決定要準備哪一組單位陣列
    String[] units;
    if ("長度".equals(selectedType)) {
        units = new String[]{"公尺", "公分", "英吋", "英尺"};
    } else if ("重量".equals(selectedType)) {
        units = new String[]{"公斤", "公克", "磅", "盎司"};
    } else { // 溫度
        units = new String[]{"攝氏", "華氏", "克氏"};
    }

    // 4. 把剛剛選好的單位陣列透過迴圈加入到上下兩個 JComboBox 中
    for (String u : units) {
        cbSource.addItem(u);
        cbTarget.addItem(u);
    }
```

```
        // 5. 預設讓目標單位的選單跳到第二個選項（例如從公尺轉成公分，比較符合直覺）
        if (cbTarget.getItemCount() > 1) {
            cbTarget.setSelectedIndex(1);
        }
    }
```

- **為什麼需要它？：**
  因為規格要求「切換換算類型時，兩個單位 JComboBox 的選項要跟著換」。當使用者把大分類從「長度」切換成「重量」時，下方單位就要從（公尺/公分...）變成（公斤/公克...），這就是 `updateUnits` 負責做的事。 image_0ab81c.png

3. `convert` 方法

這是負責**數學換算與計算邏輯**的自訂方法。當使用者輸入數字、選好單位並點擊「換算」按鈕後，程式就會把資料傳給 `convert` 來計算結果。

- **運作原理（以「長度」為例）：**
  不同單 位直接轉換很複雜（例如從英吋直接轉英尺），所以程式通常會採取「先轉成統一的基準單位，再轉成目標單位」的策略：
  1. **步驟一（來源 → 基準）**：把使用者輸入的數值，先統一換算成基本的「公尺」。
     - 如果使用者選的是「公分」，就把數值乘上 `0.01` 變回公尺。
     - 如果選的是「英吋」，就把數值乘上 `0.0254` 變回公尺。
  2. **步驟二（基準 → 目標）**：把剛剛算好的「公尺」，除以目標單位的換算比例，得到最終答案。
- **溫度為什麼比較特別？：**
  溫度（攝氏、華氏、克氏）不能單純用乘除法，所以程式裡是用公式來轉換（例如華氏轉攝氏公式：$(val - 32) \times 5/9$），最後再轉成目標溫度。